

Parallel OpenSees Applications for Developers

Frank McKenna
UC Berkeley

OpenSees Parallel Workshop
Berkeley, CA

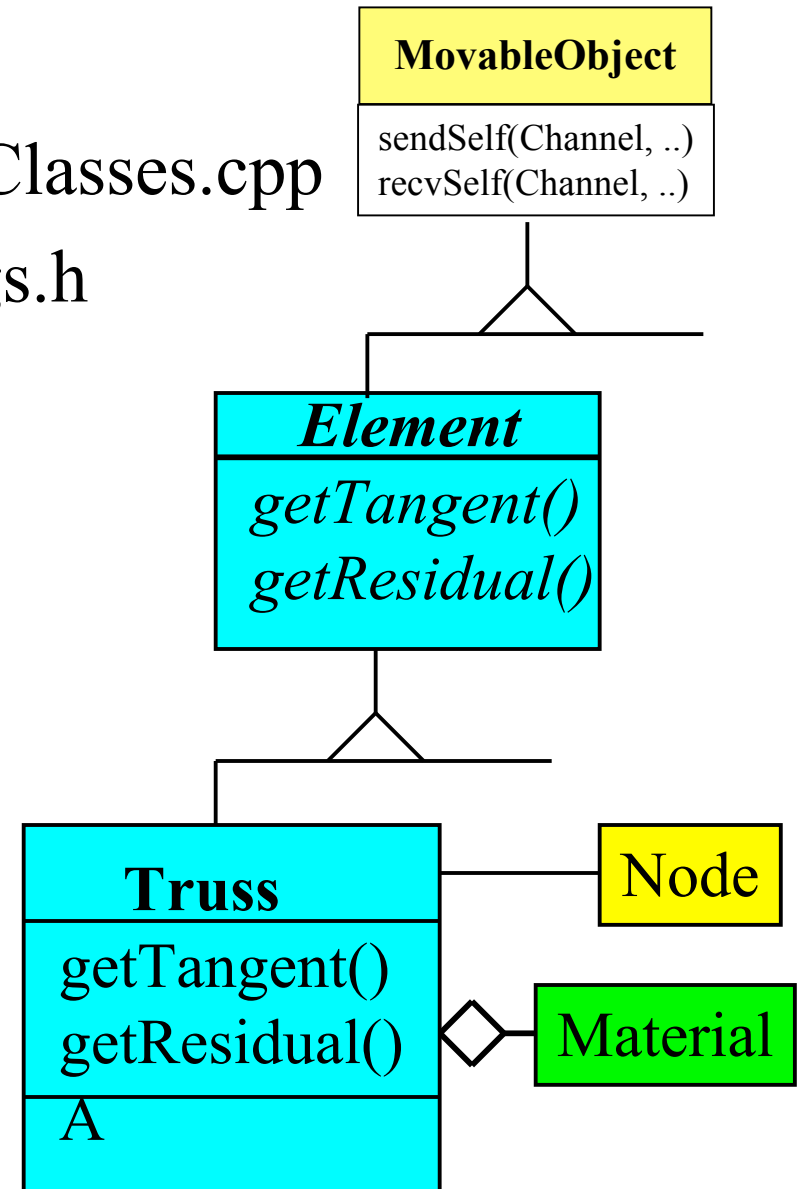


Outline

- How to Extend/Modify the Parallel Applications
- This will be **BRIEF!**

Introducing New Classes

1. Implement send/recvSelf
2. Modify FEMObjectBrokerAllClasses.cpp
3. Add classTag to SRC/classTags.h



sendSelf()

```
int
Truss::sendSelf(int commitTag, Channel &theChannel)
{
    int res = 0;

    static Vector data(7);
    data(0) = this->getTag();
    data(1) = dimension;
    data(2) = numDOF;
    data(3) = A;
    data(6) = rho;
    data(4) = theMaterial->getClassTag();
    data(5) = theMaterial->getDbTag();

    res += theChannel.sendVector(dataTag, commitTag, data);

    res += theChannel.sendID(dataTag, commitTag, connectedExternalNodes);

    res = theMaterial->sendSelf(commitTag, theChannel);
    if (res < 0) {
        opserr << "WARNING Truss::sendSelf() - " << this->getTag() << " failed to send its Material\n";
        return -3;
    }

    return 0;
}
```

recvSelf()

```
int
Truss::recvSelf(int commitTag, Channel &theChannel, FEM_ObjectBroker &theBroker)
{
    int res =0;
    int dataTag = this->getDbTag();

    static Vector data(7);
    res = theChannel.recvVector(dataTag, commitTag, data);

    this->setTag((int)data(0));
    dimension = (int)data(1);
    numDOF = (int)data(2);
    A = data(3);
    rho = data(6);

    res += theChannel.recvID(dataTag, commitTag, connectedExternalNodes);

    int matClass = (int)data(4);
    int matDb = (int)data(5);

    theMaterial = theBroker.getNewUniaxialMaterial(matClass);
    theMaterial->setDbTag(matDb);

    res += theMaterial->recvSelf(commitTag, theChannel, theBroker);
    if (res < 0) {
        opserr <<"WARNING Truss::recvSelf() - "<< this->getTag() << "failed to receive its Material\n";
        return -3;
    }

    return 0;
}
```

classTags.h

```
#define ELE_TAG_Truss 4001
```

```
.....
```

- *Just make sure it is a unique number among element classes*

FEM_ObjectBrokerAllClasses.cpp

```
Element      *  
FEM_ObjectBrokerAllClasses::getNewElement(int classTag)  
{  
    switch(classTag) {  
  
        case ELE_TAG_Truss:  
            return new Truss();  
  
        case ELE_TAG_TrussSection:  
            return new TrussSection();  
        .....  
    }
```

OpenSeesSP - SRC/tcl/mpiMain.cpp

```
main(int argc, char **argv)
{
    FEM_ObjectBrokerAllClasses theBroker;
    MPI_MachineBroker theMachine(&theBroker, argc, argv);
    theMachineBroker = &theMachine;

    int rank = theMachine.getPID();
    int np = theMachine.getNP();

    if (rank != 0) {

        // on slave processes we spin waiting to create & run actors

        fprintf(stderr, "Slave Process Running %d\n", rank);
        theMachine.runActors();

    } else {

        // on process 0 we create some ShadowSubdomains & then start the OpenSees interpreter

        fprintf(stderr, "Master Process Running OpenSees Interpreter %d\n", rank);
        g3TclMain(argc, argv, TCL_LOCAL_APPINIT, 1, 0);
        theMachine.shutdown();
    }
}
```


Modifying the Parallel Applications

- Write the code for the procedure or class.
- Place it in the main procedure of the application or in `commands.cpp`
- Use the processor defs
 1. `_PARALLEL`
 2. `_PARALLEL_INTERPRETERS`

OpenSeesMP -

SRC/tcl/mpiParameterMain.cpp

```
main(int argc, char **argv)
{
    FEM_ObjectBrokerAllClasses theBroker;
    MPI_MachineBroker theMachine(&theBroker, argc, argv);
    theMachineBroker = &theMachine;

    int rank = theMachine.getPID();
    int np = theMachine.getNP();

    g3TclMain(argc, argv, TCL_LOCAL_APPINIT, np, rank);
    theMachine.shutdown();
}
```

SRC/tcl/commands.cpp

- Most of parallel commands/modifications can be found in commands.cpp
- OpenSeesSP `#ifdef _PARALLEL_PROCESSING`

```
int analyzeModel(ClientData clientData, Tcl_Interp *interp, int argc, TCL_Char **argv)
{
    #ifdef _PARALLEL_PROCESSING
    if (OPS_PARTITIONED == false && OPS_NUM_SUBDOMAINS > 1)
        if (partitionModel() < 0) {
            opserr << "WARNING before analysis; partition failed\n";
            return TCL_ERROR;
        }
    #endif
}
```

- OpenSeesMP `#ifdef _PARALLEL_INTERPRETERS`

```
int opsBarrier(ClientData clientData, Tcl_Interp *interp, int argc, TCL_Char **argv)
{
    #ifdef _PARALLEL_INTERPRETERS
    return MPI_Barrier(MPI_COMM_WORLD);
    #endif

    return TCL_OK;
}
```

Write Your Own Parallel Applications

- Start with an existing one!

Any Questions?